# Post Implementation Review: First-Class Traits (Stage 3)

Prologos

2026-03-09

## Contents

**1 Overview** **2**

**2 1. Objectives and Scope** **3**
2.1 1.1 Original Goal . . . . . 3
2.2 1.2 Planned Scope . . . . . 3
2.3 1.3 Actual Scope . . . . . 4

**3 2. What Was Delivered** **4**
3.1 2.1 Quantitative Summary . . . . . 4
3.2 2.2 Architectural Components Delivered . . . . . 4

**4 3. What Went Well** **5**
4.1 3.1 Single-Session Implementation . . . . . 5
4.2 3.2 Existing Infrastructure Paid Dividends (Again) . . . . . 5
4.3 3.3 The Constraint Cell Abstraction Generalized Cleanly . . . 6
4.4 3.4 Phase 3d Was Surgical . . . . . 6
4.5 3.5 Zero Regressions . . . . . 6

**5 4. What Was Challenging** **7**
5.1 4.1 Constraint Chain Reader Integration . . . . . 7
5.2 4.2 WS-Mode Gaps in HasMethod Syntax . . . . . 7
5.3 4.3 Narrowing Through Complex DTs . . . . . 7

**6 5. How Results Compared to Expectations** **8**
6.1 5.1 Timeline: Faster Than Expected . . . . . 8
6.2 5.2 Test Count: Almost Exactly As Expected . . . . . 8

6.3 5.3 Scope: Clean Completion . . . . . 8
6.4 5.4 Architecture: Constraint Cell Generalization Was the Right Call . . . . . 8

**7 6. Demo Issue Catalogue 9**
7.1 Issue 1: String `add` resolves to Nat FQN (Line 76) . . . . . 9
7.2 Issue 2: Multiplication/subtraction narrowing returns nil (Lines 114, 118) . . . . . 9
7.3 Issue 3: Wrapper function narrowing over-approximates (Lines 216, 219, 336) . . . . . 9
7.4 Issue 4: String pipe type error (Line 252) . . . . . 10
7.5 Issue 5: Recursive `even?` narrowing returns nil (Line 284) . . 10
7.6 Issue 6: `compare` and `eq?` are unbound (Lines 303, 306, 310, 313) . . . . . 10
7.7 Issue 7: `collatz-step` fails to parse (Lines 463–476) . . . . . 10

**8 7. Architectural Validation 11**
8.1 7.1 Propagator-as-Universal-Substrate: Confirmed (Third Time) 11
8.2 7.2 AST Pipeline Extension Points . . . . . 11
8.3 7.3 Narrowing Engine Integration . . . . . 12

**9 8. Forward Enablement 12**
9.1 8.1 WS Surface for HasMethod (Immediate) . . . . . 12
9.2 8.2 Parameterized Constraints (Medium-Term) . . . . . 12
9.3 8.3 Narrowing Improvements (Informed by Demo Issues) . . . 12
9.4 8.4 Trait Method Re-export (Informed by Demo Issue 6) . . . 13

**10 9. Lessons Learned 13**
10.1 9.1 Technical Lessons . . . . . 13
10.2 9.2 Process Lessons . . . . . 14

**11 10. Comparison with Prior PIRs 14**
11.1 10.1 Session Types PIR (2026-03-04) vs. First-Class Traits PIR 14

**12 11. Conclusion 15**

# 1 Overview

| Field | Value |
|---|---|
| **Date** | 2026-03-09 |
| **Scope** | Phases 1–3d of the First-Class Traits feature track (Stage 3) |
| **Duration** | ~7 hours (2026-03-09 16:13 through 22:52) |
| **Design Duration** | ~11 hours (2026-03-09 04:52 through 15:32, design + critique) |
| **Commits** | Phase 1 (`f6b540e`) through capstone (`481e6de`) — 19 total |
| **Design Document** | `docs/tracking/2026-03-09_FIRST_CLASS_TRAITS_STAGE3_DESIGN.md` |
| **Capstone Demo** | `examples/2026-03-09-fc-trait-rel-dom.prologos` |

---

# 2 1. Objectives and Scope

## 2.1 1.1 Original Goal

Make traits first-class in Prologos across three cumulative levels:

| Level | What becomes first-class | Key capability |
|---|---|---|
| 1 | Trait names as type constructors | Abstract over *which trait* at the type level |
| 2 | Trait dispatch as propagator cells | Bidirectional typedispatch; narrowing through ops |
| 3 | Traits as first-class propositions | Quantify over traits; method projection; relational constr |

The central thesis: traits are constraints on types. Making constraints first-class creates a combinatorial surface where functional code, relational narrowing, and type-level dispatch compose freely.

## 2.2 1.2 Planned Scope

9 phases across 3 levels:

| # | Phase | Level | Deliverable |
|---|---|---|---|
| 1 | Phase 1 | L1 | Trait names as type constructors (`trait-ref`) |
| 2 | Phase 2a | L2 | Static generic operator resolution in narrowing |
| 3 | Phase 2b | L2 | `constraint-cell.rkt` finite-domain lattice |
| 4 | Phase 2c | L2 | Constraint propagators P1–P4 |
| 5 | Phase 2d | L2 | ATMS multi-candidate search |
| 6 | Phase 3a | L3 | HasMethod constraint + method projection |
| 7 | Phase 3b | L3 | Trait introspection (instances-of, methods-of, satisfies?) |
| 8 | Phase 3c | L3 | Constraint chain syntax `?var:C1:C2` |
| 9 | Phase 3d | L3 | Incremental trait resolution (no batch pass) |

### 2.3 1.3 Actual Scope

**9 of 9 phases completed. Scope adherence: 100%.**

No phases were deferred and no unplanned scope was added. The design document's phase ordering was followed exactly. The only out-of-order decision was implementing Phase 3a (HasMethod) before Phase 2c–2d, because HasMethod's `:over/:method` spec syntax was independent of the propagator infrastructure and could be tested in sexp mode immediately.

---

## 3 2. What Was Delivered

### 3.1 2.1 Quantitative Summary

| Metric | Value |
|---|---|
| New implementation files | 2 (constraint-cell, constraint-propagators) |
| New implementation LOC | 487 (195 + 292) |
| Modified source files (significantly) | 16 (across the AST pipeline) |
| Total implementation changes | 1,367 insertions, 117 deletions |
| New test files | 8 |
| New test LOC | 1,612 |
| Test cases added | 391 |
| Overall test suite growth | 6,290 → 6,681 (+391, +6.2%) |
| Test files growth | 337 → 345 (+8) |
| Capstone demo | 518 lines, 15 sections, ~45 live expressions |
| Implementation commits | 9 (phases) + 8 (tracker updates) + 1 (demo) + 1 (dep-graph) = |
| Design commits | 4 (design doc, critique, refinement, tracker) |

### 3.2 2.2 Architectural Components Delivered

1. `trait-ref` **type constructor** — Trait names (`Add`, `Eq`, `Ord`) usable in type-level positions, enabling abstraction over which trait is satisfied.

2. **Generic operator resolution in narrowing** — `add`, `sub`, `mult`, `div` in narrowing queries resolve through the trait registry, extract concrete definitional trees, and run the DT backwards.

3. `constraint-cell.rkt` — Pure finite-domain lattice (bot/set/one/top) with merge = set intersection. Domain-generic: works for dispatch candidates, type memberships, predicate constraints, and interval constraints.

4. **Constraint propagators P1–P4** — Four propagators maintaining consistency across type cells, dispatch cells, and method evidence. Registry-based dispatch replaces the static operator table.

5. **ATMS multi-candidate search** — When type inference hasn't resolved a unique trait impl, enumerate candidates in ATMS worldviews, run narrowing in each, and merge results with deduplication.

6. **HasMethod constraint + method projection** — `:over/:method` spec metadata for abstracting over which trait provides a method. Search-based trait P discovery via the trait registry.

7. **Trait introspection** — Three REPL/sexp commands: `instances-of`, `methods-of`, `satisfies?`. Query the trait registry at runtime.

8. **Constraint chain syntax** `?var:C1:C2` — Reader greedy consumption of `:UpperCase` segments, parser constraint splitting, narrowing type-guard forward-checking.

9. **Incremental trait resolution** — Removed all batch `resolve-trait-constraints!` calls from the pipeline. Trait constraints resolved immediately when type-arg metas are solved, via `solve-meta!` → `retry-trait-for-meta!` callback chain.

---

# 4 3. What Went Well

## 4.1 3.1 Single-Session Implementation

All 9 phases were designed, implemented, tested, and committed in a single working session (~7 hours of implementation, ~11 hours of design beforehand). The design document's thoroughness — each phase had clear scope, file list, and test targets — eliminated ambiguity during coding.

## 4.2 3.2 Existing Infrastructure Paid Dividends (Again)

The pattern from the Session Types PIR repeats: mature infrastructure absorbs new features without modification.

- **Propagator network** (`propagator.rkt`): zero changes. Constraint propagators used `net-add-propagator` / `run-to-quiescence` unchanged.

- **ATMS** (`atms.rkt`): zero changes. Multi-candidate search used existing worldview enumeration.
- **Narrowing engine** (`narrowing.rkt`): minimal changes (11 insertions). The DT-guided search needed only a hook for trait-resolved operator dispatch.
- **Trait registry** (`trait-resolution.rkt`): extended with query functions but core resolution logic unchanged.

### 4.3 3.3 The Constraint Cell Abstraction Generalized Cleanly

The design document proposed `dispatch-lattice.rkt` specifically for trait impl candidate sets. During implementation, this was generalized to `constraint-cell.rkt` — a domain-generic finite-domain lattice. The same structure handles dispatch candidates, type memberships, predicate constraints, and interval constraints. The generalization was free: the lattice operations (bot, set, one, top, merge = intersection) don't depend on the element type.

### 4.4 3.4 Phase 3d Was Surgical

Removing the batch `resolve-trait-constraints!` post-pass — the most architecturally significant change — required modifying only 2 files:

- `metavar-store.rkt`: 8 lines added (immediate-resolve for already-ground type-args)
- `expander.rkt`: 4 call sites removed (replaced with comments)

The incremental resolution infrastructure had been built across prior sprints (Sprint 5–9) and the FL Narrowing track. Phase 3d merely removed the redundant batch fallback. This validates the architectural principle: when the incremental path is correct, the batch path is pure overhead.

### 4.5 3.5 Zero Regressions

All 6,681 tests pass after every phase. No existing test broke during the entire feature track. The new infrastructure is purely additive — it doesn't modify existing dispatch or resolution behavior, only adds new entry points.

---

# 5 4. What Was Challenging

## 5.1 4.1 Constraint Chain Reader Integration

Phase 3c's `?var:C1:C2` syntax required changes at three pipeline stages:

- **Reader** (`reader.rkt`): Greedy consumption of `:UpperCase` segments into a single token. The reader must distinguish `?x:Nat` (constraint chain) from `?x:nat` (lowercase, not a constraint) and `?x :` expr= (assignment, not a constraint).
- **Parser** (`parser.rkt`): Split the combined token into variable name + constraint list.
- **Surface syntax** (`surface-syntax.rkt`): New `surf-constrained-lvar` struct.

The greedy reader approach is simple but means constraint chains can't contain lowercase identifiers (e.g., `?x:myType` won't parse as a constraint). This is consistent with the naming convention (types are UpperCase) but is a design trade-off worth noting.

## 5.2 4.2 WS-Mode Gaps in HasMethod Syntax

Phase 3a's `:over/:method` spec metadata works in sexp mode but has no WS surface syntax. The WS preparser's spec metadata parser (`macros.rkt`) would need extension to recognize these keywords in whitespace-significant mode. This was intentionally deferred (tracked in DEFERRED.md) as a WS integration task, but it means the capstone demo can only show HasMethod as commented ideal syntax.

## 5.3 4.3 Narrowing Through Complex DTs

The capstone demo revealed that backward narrowing through wrapper functions and nested pattern matches doesn't always produce expected results. See §6 (Demo Issue Catalogue) for details. These aren't Stage 3 bugs per se — they're limitations of the narrowing engine's DT-guided search that Stage 3's trait dispatch exposes more visibly.

---

# 6 5. How Results Compared to Expectations

## 6.1 5.1 Timeline: Faster Than Expected

The design document suggested a multi-session implementation. Actual: single session. Contributing factors:

- Mature infrastructure: no new lattice/propagator primitives needed
- Phase 1 established the pattern; subsequent phases were variations
- The out-of-order Phase 3a decision (HasMethod before propagators) parallelized the conceptual work: sexp-mode spec metadata is independent of narrowing

## 6.2 5.2 Test Count: Almost Exactly As Expected

391 new test cases across 8 test files. The distribution:

| Test File | Cases |
|---|---:|
| test-constraint-cell-01.rkt | 99 |
| test-constraint-chain-01.rkt | 59 |
| test-constraint-propagators-01.rkt | 56 |
| test-trait-introspection-01.rkt | 56 |
| test-trait-tycon-01.rkt | 45 |
| test-hasmethod-01.rkt | 34 |
| test-constraint-amb-01.rkt | 26 |
| test-trait-narrowing-01.rkt | 16 |
| **Total** | **391** |

## 6.3 5.3 Scope: Clean Completion

No phases deferred, no scope creep. The design document's 9-phase structure matched the implementation exactly. The only structural deviation was phase ordering (3a before 2c–2d), which was a beneficial optimization, not a deviation.

## 6.4 5.4 Architecture: Constraint Cell Generalization Was the Right Call

The design document's transition from `dispatch-lattice` to `constraint-cell` (§4.1) proved correct. The same module supports dispatch cells, and the API

is clean enough that future constraint types (interval, predicate) can plug in without modification.

---

# 7 6. Demo Issue Catalogue

The capstone demo (`examples/2026-03-09-fc-trait-rel-dom.prologos`) surfaced 7 categories of issues when evaluated from a source-connected REPL. These are catalogued here with hypotheses; none are addressed in this PIR.

## 7.1 Issue 1: String `add` resolves to Nat FQN (Line 76)

**Symptom**: `[add "hello " "world"]` returns `nil` instead of `"hello world"`.

**Hypothesis**: The expression evaluates as `[prologos::data::nat::add "hello " "world"]` (per the error trace). The generic `add` is resolving to the Nat instance's FQN instead of String's `String--Add--dict`. This may be a prelude import ordering issue: the Nat-specific `add` function (non-generic, defined in `prologos::data::nat`) shadows the generic `add` trait method.

## 7.2 Issue 2: Multiplication/subtraction narrowing returns nil (Lines 114, 118)

**Symptom**: `mult ?x ?y = 6N` and `sub ?x ?y = 3N` return `nil` or incomplete results.

**Hypothesis**: Nat multiplication is defined via recursive addition, creating a DT with deep nesting. The backward search exhausts its fuel (default depth limit) before finding factor pairs. Saturating subtraction (`sub 3N 5N = 0N`) creates a non-injective DT that the backward search over-approximates. These are narrowing engine depth/shape limitations, not Stage 3 bugs.

## 7.3 Issue 3: Wrapper function narrowing over-approximates (Lines 216, 219, 336)

**Symptom**: `[my-double ?x] = 6N` returns `'[{:x 6N} {:x 5N} {:x 4N} {:x 3N}]` instead of `'[{:x 3N}]`.

**Hypothesis**: `my-double` is defined as `[add n n]`, creating a trivial DT that wraps `add`. When narrowing backwards, the engine decomposes to `add ?n ?n = 6N` but treats the two `?n` occurrences as independent variables (no aliasing constraint). The DT search finds all `(x, y)` pairs where `x +`

`y = 6`, not just the diagonal `x = y`. This is a known limitation of DT-guided narrowing for non-linear patterns — aliased variables require either equational unification or a post-filter.

### 7.4 Issue 4: String pipe type error (Line 252)

**Symptom**: `"hello" |> [add " "] |> [add "world"]` produces a type error referencing `prologos::data::nat::add`.

**Hypothesis**: Same root cause as Issue 1 — generic `add` resolves to the Nat-specific function rather than the String trait instance. The pipe merely exposes it in a different context. The partially-applied `[add " "]` elaborates with the Nat FQN, then fails when applied to a String argument.

### 7.5 Issue 5: Recursive `even?` narrowing returns nil (Line 284)

**Symptom**: `[even? ?n:Nat] = true` returns `nil`.

**Hypothesis**: `even?` has a nested match (match-within-match) that creates a complex DT with recursive calls. The backward search either: (a) exceeds depth for the recursive `even?` call in the `suc (suc j)` branch, or (b) the nested match generates a DT shape that the backward walker doesn't decompose correctly. Nested match DT extraction is a known area for improvement.

### 7.6 Issue 6: `compare` and `eq?` are unbound (Lines 303, 306, 310, 313)

**Symptom**: `Unbound variable: compare` and `Unbound variable: eq?`.

**Hypothesis**: The prelude's `:refer` lists for `prologos::core::ord` and `prologos::core::eq` export wrapper functions (`ord-lt`, `eq-check`, `nat-eq`) but **not** the trait method names (`compare`, `eq?`) themselves. The trait method names are defined inside trait bodies and are only accessible through dict dispatch, not as standalone functions. To use them, the user must either: (a) call the wrapper (`eq-check`, `ord-compare`), or (b) import the method explicitly. The capstone demo assumed these were prelude-exported generic functions.

### 7.7 Issue 7: `collatz-step` fails to parse (Lines 463–476)

**Symptom**: `Unbound variable: spec` and `defn requires: ...` errors.

**Hypothesis**: The `spec=/=defn` pair at lines 463–464 is being parsed in an unexpected context. Possible causes: (a) A prior expression's evaluation changed the parser state (e.g., the file-level `ns` form expects all subsequent forms to be in expression mode after the narrowing queries). (b) The WS-mode file processor treats narrowing queries (`add ?x ?y = 5N`) as expressions, and after processing them, the parser context may not reset to accept new declaration forms. (c) There may be a scoping interaction where the prior `even?` definition's recursive DT leaves the parser in a state that doesn't recognize subsequent `spec` forms.

---

# 8 7. Architectural Validation

## 8.1 7.1 Propagator-as-Universal-Substrate: Confirmed (Third Time)

Stage 3 is the fourth domain using the propagator network:

| Domain | Cells | Propagators | Committed |
|---|---|---|---|
| Type inference | Type lattice cells | Unification propagators | Sprint 5 |
| QTT | Multiplicity cells | mult-propagators | Sprint 7 |
| Session types | Session lattice cells | Send/recv/duality propagators | S4 |
| **Trait dispatch** | **Constraint cells (dispatch)** | **P1–P4 (typeconstraintmethod)** | **Stage 3** |

Zero changes to `propagator.rkt` across all four domains. The abstraction holds.

## 8.2 7.2 AST Pipeline Extension Points

Stage 3 touched 14 of the 14 AST pipeline files (syntax → surface → parser → elaborator → typing-core → qtt → reduction → substitution → zonk → pretty-print + unify, macros, foreign). The extension points — adding new struct types to `syntax.rkt` and `surface-syntax.rkt`, adding elaboration cases to `elaborator.rkt`, adding reduction rules to `reduction.rkt` — all worked as expected.

The one friction point: `macros.rkt` (WS desugaring) is 7,600+ lines and growing. Spec metadata parsing for new keywords (`:over`, `:method`) required understanding a 1,200-line function. This is not a correctness issue but an ergonomics issue for future developers.

### 8.3 7.3 Narrowing Engine Integration

The narrowing engine (`narrowing.rkt`) required only 11 lines of changes — a hook for trait-resolved operator dispatch. This validates the DT-based architecture: trait dispatch is orthogonal to the search algorithm. The narrowing engine doesn't know or care about traits; it receives concrete DTs after dispatch resolution and walks them backwards as before.

However, the demo issues (§6) reveal that this clean separation has a cost: the narrowing engine can't exploit trait-level information (e.g., knowing that `add` is commutative) for search pruning. The DT is the only interface, and DTs don't carry algebraic properties.

---

## 9 8. Forward Enablement

### 9.1 8.1 WS Surface for HasMethod (Immediate)

Phase 3a's `:over/:method` works in sexp mode. Wiring up WS surface syntax is a straightforward parser extension — add keyword recognition to `parse-spec-metadata` in the WS preparser path. This would complete the capstone demo's §12.

### 9.2 8.2 Parameterized Constraints (Medium-Term)

The design document specifies `?n:Between[50 100]:Even` syntax with parameterized constraints. The reader already handles `?var:C` chains; extending to `C[args]` requires a small reader change (recognize `[` after a constraint identifier). The `constraint-cell.rkt` lattice is domain-generic and handles parameterized constraints without modification.

### 9.3 8.3 Narrowing Improvements (Informed by Demo Issues)

The demo issues point to three specific narrowing improvements:

1. **Variable aliasing in DT search** (Issue 3): When a function uses a variable non-linearly (`[add n n]`), the backward search should unify the alias occurrences, not treat them as independent. This requires equational constraints in the DT walk.

2. **Depth-bounded multiplication/subtraction** (Issue 2): Deeper fuel or iterative deepening for recursive DTs.

3. **Nested match DT extraction** (Issue 5): Match-within-match should produce composed DTs, not flat ones.

### 9.4 8.4 Trait Method Re-export (Informed by Demo Issue 6)

The prelude should export trait method names (`eq?`, `compare`) as generic functions, not just wrapper functions. This is a `namespace.rkt` change — add the trait method names to the `:refer` lists for `prologos::core::eq` and `prologos::core::ord`.

---

## 10 9. Lessons Learned

### 10.1 9.1 Technical Lessons

1. **Constraint cells generalize dispatch cells.** Starting with a domain-specific `dispatch-lattice` and generalizing to `constraint-cell` during implementation was the right move. The generalization was free and enables future constraint types.

2. **Registry-based dispatch > static operator table.** Phase 2c replaced a static table mapping operators to trait names with a dynamic registry lookup. This means user-defined traits automatically participate in narrowing dispatch without code changes.

3. **Immediate-resolve safety net for ground type-args.** In `register-trait-constraint!`, if all type-args are already ground (no metas), the wakeup map has no entries and incremental resolution would never fire. The safety net (Phase 3d) invokes the resolve callback immediately. Without this, already-ground constraints would silently fail.

4. **Prelude export lists are a user-facing API.** The demo's `compare=/=eq?` unbound errors (Issue 6) show that what's in the `:refer` list is what users can call. Trait method names should be treated as first-class exports, not internal implementation details.

5. **Non-linear DT variables need equational unification.** The `my-double` over- approximation (Issue 3) reveals a fundamental gap: DT-guided backward search treats each variable occurrence independently. Functions with aliased variables (`[add n n]`) need either: (a) a post-filter

unifying alias groups, or (b) equational constraints threaded through the DT walk.

### 10.2 9.2 Process Lessons

1. **Phase ordering flexibility matters.** The design document numbered phases sequentially (1, 2a, 2b, 2c, 2d, 3a, 3b, 3c, 3d) but implementation went 1, 2a, 2b, 3a, 2c, 2d, 3b, 3c, 3d. The out-of-order Phase 3a was beneficial: HasMethod's spec metadata was independent of propagator infrastructure and could be tested immediately.

2. **Capstone demos are diagnostic instruments.** The demo surfaced 7 issue categories that unit tests didn't catch — not because the unit tests are wrong, but because the demo exercises end-to-end composition in ways that unit tests don't. The narrowing engine passes all its tests; the trait dispatch passes all its tests; but their *composition* in a user-facing `.prologos` file reveals gaps.

3. **Single-session features benefit from design doc investment.** The ~11 hours of design (research, external critique, refinement) paid off in ~7 hours of clean implementation. Design:implementation ratio of ~1.6:1 is efficient for a feature of this complexity.

---

## 11 10. Comparison with Prior PIRs

### 11.1 10.1 Session Types PIR (2026-03-04) vs. First-Class Traits PIR

| Dimension | Session Types | First-Class Traits |
|---|---:|---:|
| Phases | 36/38 (95%) | 9/9 (100%) |
| Duration (impl) | ~24 hours | ~7 hours |
| New impl files | 4 | 2 |
| New impl LOC | 1,880 | 487 |
| Modified files | 9 | 16 |
| Total changes | ~23,500 | 1,484 |
| New test files | 27 | 8 |
| New test cases | 392 | 391 |
| Suite growth | +21% | +6.2% |
| `propagator.rkt` changes | 0 | 0 |
| Regressions | 0 | 0 |

Key observation: Stage 3 was smaller in raw LOC but wider in file touches (16 vs. 9). This reflects the nature of the work — session types added new infrastructure files; first-class traits threaded new dispatch paths through *existing* infrastructure. The near-identical test case count (392 vs. 391) suggests a stable testing discipline.

---

## 12 11. Conclusion

The First-Class Traits feature track achieved its objectives: Prologos traits are now first-class citizens that can be stored, passed, inspected, and used as constraint domains on logic variables. The implementation validated the propagator-as-universal- substrate architecture for the fourth time and demonstrated that the constraint cell lattice generalizes beyond trait dispatch.

The capstone demo validated the composition thesis — functional code, relational narrowing, and type-level dispatch compose naturally — while surfacing 7 issue categories that inform future narrowing improvements. The most significant finding is that non-linear variable aliasing in DT-guided backward search is the primary gap preventing full compositional narrowing.

**Final metrics**: 19 commits, 9 completed phases, 487 new implementation LOC + 997 changed LOC, 391 test cases, zero regressions, zero deferrals.